TUGAS M5 - DATA MINING

VALIDATION MODEL OF CLASSIFICATION

GUSTHI PANGESTU

D4 IT D | 3124600098

Tugas 4 > jawaban > 01_load_dataset.py > ...

```python
import os
import pandas as pd

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'

dataset = pd.read_csv(csv_path)

print("=== 1. Load Dataset Train (titanic.csv) ===")
print(f"File Path : {csv_path}")
print(f"Dimensi   : {dataset.shape[0]} baris, {dataset.shape[1]} kolom\n")

print("--- Sampel 5 Baris Data Acak (Random Sample) ---")
print(dataset.sample(5, random_state=42))

print("\n--- Ringkasan Kolom ---")
print(dataset.info())
```

1. LOAD DATASET

Load Dataset ***titanic.csv*** terdapat 891 baris dan 12 kolom menggunakan Pandas, lalu mengeksplorasi sampel acak 5 baris data serta struktur kolomnya di mana saya menemukan adanya nilai kosong (missing values) pada fitur Age, Cabin, dan Embarked yang perlu dibersihkan pada tahap selanjutnya.

HASIL LOAD DATASET

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban> python 01_load_dataset.py
=== 1. Load Dataset Train (titanic.csv) ===
File Path : C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban\..\data\titanic.csv
Dimensi   : 891 baris, 12 kolom

--- Sampel 5 Baris Data Acak (Random Sample) ---
     PassengerId  Survived  Pclass                                               Name     Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
709          710         1       3  Moubarek, Master. Halim Gonios ("William George")    male   NaN      1      1              2661  15.2458   NaN        C
439          440         0       2            Kvillner, Mr. Johan Henrik Johannesson    male  31.0      0      0       C.A. 18723  10.5000   NaN        S
840          841         0       3                      Alhomaki, Mr. Ilmari Rudolf    male  20.0      0      0  SOTON/O2 3101287   7.9250   NaN        S
720          721         1       2                Harper, Miss. Annie Jessie "Nina"  female   6.0      0      1           248727  33.0000   NaN        S
39            40         1       3                      Nicola-Yarred, Miss. Jamila  female  14.0      1      0             2651  11.2417   NaN        C

--- Ringkasan Kolom ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

Tugas 4 > jawaban > 02_validation_methods.py > ...

```python
import os
import pandas as pd
from sklearn.model_selection import KFold, LeaveOneOut, train_test_split

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'

dataset = pd.read_csv(csv_path)
target = "Survived"

print("=== 2. Skema Pembagian Metode Validasi Model ===")
print(f"Total Dataset: {len(dataset)} baris\n")

# 2.a Hold-out Method (70% Train, 30% Test)
print("--- 2.a Hold-out Method (70% Train, 30% Test) ---")
train_df, test_df = train_test_split(
    dataset, test_size=0.30, random_state=42, stratify=dataset[target]
)
print(f"Jumlah Data Train (70%) : {len(train_df)} baris")
print(f"Jumlah Data Test  (30%) : {len(test_df)} baris")
print(f"Distribusi Target Train : {dict(train_df[target].value_counts())}")
print(f"Distribusi Target Test  : {dict(test_df[target].value_counts())}\n")

# 2.b K-Fold Cross Validation (k = 10)
print("--- 2.b K-Fold Cross Validation (k = 10) ---")
kf = KFold(n_splits=10, shuffle=True, random_state=42)
print(f"Jumlah Lipatan (Folds)  : {kf.get_n_splits(dataset)}")
for fold, (train_idx, test_idx) in enumerate(kf.split(dataset), 1):
    print(f"  Fold {fold:02d} -> Train: {len(train_idx)} baris, Test: {len(test_idx)} baris")
print()

# 2.c Leave-One-Out (LOO) Cross Validation
print("--- 2.c Leave-One-Out (LOO) Cross Validation ---")
loo = LeaveOneOut()
n_splits = loo.get_n_splits(dataset)
print(f"Total Eksperimen (N)    : {n_splits}")
print("Setiap iterasi          : Data Train = N - 1 (890 baris), Data Test = 1 baris")

```

2. METODE VALIDASI MODEL

Mempersiapkan dan membagi dataset ***titanic.csv*** menggunakan 3 metode validasi: ***Hold-out Method*** (70%-30%) dengan stratifikasi target, ***10-Fold Cross Validation*** dengan pengacakan (shuffle), serta ***Leave-One-Out*** (LOO) yang menguji satu sampel di setiap iterasi (N-1 data latih). Skema ini memastikan evaluasi model klasifikasi berjalan objektif tanpa data leakage.

HASIL METODE VALIDASI MODEL

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> python 02_validation_methods.py
=== 2. Skema Pembagian Metode Validasi Model ===
Total Dataset: 891 baris

--- 2.a Hold-out Method (70% Train, 30% Test) ---
Jumlah Data Train (70%) : 623 baris
Jumlah Data Test  (30%) : 268 baris
Distribusi Target Train : {0: 384, 1: 239}
Distribusi Target Test  : {0: 165, 1: 103}

--- 2.b K-Fold Cross Validation (k = 10) ---
Jumlah Lipatan (Folds)  : 10
  Fold 01 -> Train: 801 baris, Test: 90 baris
  Fold 02 -> Train: 802 baris, Test: 89 baris
  Fold 03 -> Train: 802 baris, Test: 89 baris
  Fold 04 -> Train: 802 baris, Test: 89 baris
  Fold 05 -> Train: 802 baris, Test: 89 baris
  Fold 06 -> Train: 802 baris, Test: 89 baris
  Fold 07 -> Train: 802 baris, Test: 89 baris
  Fold 08 -> Train: 802 baris, Test: 89 baris
  Fold 09 -> Train: 802 baris, Test: 89 baris
  Fold 10 -> Train: 802 baris, Test: 89 baris

--- 2.c Leave-One-Out (LOO) Cross Validation ---
Total Eksperimen (N)    : 891
Setiap iterasi          : Data Train = N - 1 (890 baris), Data Test = 1 baris
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

3. FITUR DATASET & CLEANING DATA

Tugas 4 > jawaban > 03_train_data_fitur.py > ...

```python
import os
import pandas as pd

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi kolom fitur numerik dan target
data = dataset[features + [target]].copy()

# Encoding fitur biner kategorikal 'Sex' (female: 0, male: 1)
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})

# Mengatasi missing value pada 'Fare' menggunakan nilai median
data["Fare"] = data["Fare"].fillna(data["Fare"].median())

# Salin data sebelum imputasi untuk bahan perbandingan evaluasi
data_before = data.copy()

# Hitung rata-rata usia per kelas keselamatan (Survived)
mean_age_per_class = data.groupby(target)["Age"].mean()
overall_mean_age = data["Age"].mean()

print("=== 3. Ekstraksi Fitur & Imputasi Missing Value 'Age' ===")
print(f"Fitur Terpilih: {features}")
print(f"Missing Value 'Age' Sebelum Imputasi : {data['Age'].isna().sum()} baris")
print(f"Rata-rata Usia Kelas Tidak Selamat (0): {mean_age_per_class[0]:.2f} tahun")
print(f"Rata-rata Usia Kelas Selamat (1)      : {mean_age_per_class[1]:.2f} tahun")
```

```python

# Isi missing value kolom Age dengan mean kelas masing-masing
for cls in mean_age_per_class.index:
    mask = (data[target] == cls) & (data["Age"].isna())
    data.loc[mask, "Age"] = mean_age_per_class[cls]

# Nilai rata-rata keseluruhan sebagai fallback jika ada kelas tak terdefinisi
data["Age"] = data["Age"].fillna(overall_mean_age)

print(f"Missing Value 'Age' Setelah Imputasi  : {data['Age'].isna().sum()} baris")

# Ambil sampel baris yang awalnya bernilai NaN pada kolom Age untuk membandingkan sebelum vs sesudah
sample_missing_indices = data_before[data_before["Age"].isna()].sample(5, random_state=42).index

print("\n--- Sampel 5 Baris Data Sebelum Imputasi (Kondisi Nilai NaN) ---")
print(data_before.loc[sample_missing_indices])

print("\n--- Sampel 5 Baris Data Setelah Imputasi (Berdasarkan Nilai Mean Kelas) ---")
print(data.loc[sample_missing_indices])

```

Mengambil kolom fitur (***Sex***, ***Age***, ***Pclass***, ***Fare***) dan target (***Survived***), melakukan encoding biner pada fitur ***Sex***, serta menangani missing value pada fitur ***Age*** dengan pengisian nilai rata-rata (mean) dari masing-masing kelas target keselamatan penumpang.

HASIL FITUR DATASET & CLEANING DATA

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> python 03_train_d
=== 3. Ekstraksi Fitur & Imputasi Missing Value 'Age' ===
Fitur Terpilih: ['Sex', 'Age', 'Pclass', 'Fare']
Missing Value 'Age' Sebelum Imputasi : 177 baris
Rata-rata Usia Kelas Tidak Selamat (0): 30.63 tahun
Rata-rata Usia Kelas Selamat (1)      : 28.34 tahun
Missing Value 'Age' Setelah Imputasi  : 0 baris

--- Sampel 5 Baris Data Sebelum Imputasi (Kondisi Nilai NaN) ---
     Sex  Age  Pclass     Fare  Survived
82     0  NaN       3   7.7875         1
235    0  NaN       3   7.5500         0
667    1  NaN       3   7.7750         0
158    1  NaN       3   8.6625         0
347    0  NaN       3  16.1000         1

--- Sampel 5 Baris Data Setelah Imputasi (Berdasarkan Nilai Mean Kelas) ---
     Sex        Age  Pclass     Fare  Survived
82     0  28.343690       3   7.7875         1
235    0  30.626179       3   7.5500         0
667    1  30.626179       3   7.7750         0
158    1  30.626179       3   8.6625         0
347    0  28.343690       3  16.1000         1
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

4. TARGET DATASET & DISTRIBUSI KELAS

Tugas 4 > jawaban > 04_label_target.py

```python
import os
import pandas as pd

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

# Ekstraksi label target kelas biner keselamatan
label = dataset["Survived"]

print("=== 4. Ekstraksi Label Kelas Target (Survived) ===")
print(f"Kolom Target: {label.name}")
print(f"Total Data  : {len(label)} baris\n")

print("--- Distribusi Frekuensi Kelas Target ---")
distribusi = label.value_counts().sort_index()
for cls, count in distribusi.items():
    keterangan = "Tidak Selamat (0)" if cls == 0 else "Selamat (1)"
    persentase = (count / len(label)) * 100
    print(f"  Kelas {cls} ({keterangan:<17}): {count:>3} penumpang ({persentase:.2f}%)")

# Ambil sampel acak untuk melihat variasi sebaran nilai label target
sample_indices = label.sample(10, random_state=42).index

print("\n--- Sampel 10 Nilai Label Target (Acak) ---")
print(label.loc[sample_indices].to_string())

```

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawab
=== 4. Ekstraksi Label Kelas Target (Survived) ===
Kolom Target: Survived
Total Data  : 891 baris

--- Distribusi Frekuensi Kelas Target ---
  Kelas 0 (Tidak Selamat (0)): 549 penumpang (61.62%)
  Kelas 1 (Selamat (1)      ): 342 penumpang (38.38%)

--- Sampel 10 Nilai Label Target (Acak) ---
709    1
439    0
840    0
720    1
39     1
290    1
300    1
333    0
208    1
136    1
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawab
```

Mengekstrak kolom target Survived dari dataset sebagai label kelas klasifikasi biner, lalu memeriksa distribusi frekuensinya di mana terdapat 549 penumpang Tidak Selamat (0) dan 342 penumpang Selamat (1) dari total 891 baris data

5. NORMALISASI MIN-MAX PADA TRAIN DATA

Tugas 4 > jawaban > 05_normalisasi_train_minmax.py

```python
import os
import pandas as pd
from sklearn.model_selection import train_test_split

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi fitur dan target yang dibutuhkan
data = dataset[features + [target]].copy()
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})
data["Fare"] = data["Fare"].fillna(data["Fare"].median())

# Pisahkan data latih (70%) dan uji (30%) sesuai skema validasi Hold-out
train_df, test_df = train_test_split(
    data, test_size=0.30, random_state=42, stratify=data[target]
)
train_copy = train_df.copy()

# Isi missing value Age menggunakan rata-rata usia per kelas target pada data latih
mean_age_train = train_copy.groupby(target)["Age"].mean()
overall_mean_train = train_copy["Age"].mean()

for cls in mean_age_train.index:
    mask = (train_copy[target] == cls) & (train_copy["Age"].isna())
    train_copy.loc[mask, "Age"] = mean_age_train[cls]
train_copy["Age"] = train_copy["Age"].fillna(overall_mean_train)

# Ambil data fitur pelatihan yang telah bersih
train_data = train_copy[features].astype(float)
```

```python

# 5. Normalisasi Min-Max (0 - 1) & catat nilai min dan max setiap atribut
min_val = train_data.min()
max_val = train_data.max()

norm_train_data = (train_data - min_val) / (max_val - min_val)

print("=== 5. Normalisasi Min-Max 0-1 pada train_data ===")
print("Nilai Minimum Atribut Train:")
print(min_val)
print("\nNilai Maksimum Atribut Train:")
print(max_val)

# Ambil sampel acak menggunakan index yang sama untuk membandingkan sebelum vs sesudah
sample_indices = train_data.sample(5, random_state=42).index

print("\n--- Sampel 5 Baris train_data Sebelum Normalisasi (Acak) ---")
print(train_data.loc[sample_indices])

print("\n--- Sampel 5 Baris train_data Setelah Normalisasi (Acak) ---")
print(norm_train_data.loc[sample_indices])

```

Melakukan normalisasi Min-Max (rentang 0–1) khusus pada ***train_data*** setelah pemisahan data (Hold-out 70%). Pada tahap ini, nilai minimum dan maksimum dari masing-masing atribut fitur (***Sex***, ***Age***, ***Pclass***, ***Fare***) dicatat untuk nantinya digunakan sebagai acuan transformasi pada data uji.

HASIL NORMALISASI MIN-MAX PADA TRAIN DATA

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> py
=== 5. Normalisasi Min-Max 0-1 pada train_data ===
Nilai Minimum Atribut Train:
Sex       0.00
Age       0.42
Pclass    1.00
Fare      0.00
dtype: float64

Nilai Maksimum Atribut Train:
Sex         1.0000
Age        80.0000
Pclass      3.0000
Fare      512.3292
dtype: float64

--- Sampel 5 Baris train_data Sebelum Normalisasi (Acak) ---
     Sex        Age  Pclass     Fare
668  1.0  43.000000     3.0   8.0500
36   1.0  28.150097     3.0   7.2292
787  1.0   8.000000     3.0  29.1250
61   0.0  38.000000     1.0  80.0000
413  1.0  31.355705     2.0   0.0000

--- Sampel 5 Baris train_data Setelah Normalisasi (Acak) ---
     Sex       Age  Pclass      Fare
668  1.0  0.535059     1.0  0.015713
36   1.0  0.348456     1.0  0.014110
787  1.0  0.095250     1.0  0.056848
61   0.0  0.472229     0.0  0.156150
413  1.0  0.388737     0.5  0.000000
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

6. NORMALISASI TEST DATA DENGAN MIN-MAX TRAIN DATA

Tugas 4 > jawaban > 06_normalisasi_test_minmax.py > ...

```python
import os
import pandas as pd
from sklearn.model_selection import train_test_split

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi fitur dan target yang dibutuhkan
data = dataset[features + [target]].copy()
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})
data["Fare"] = data["Fare"].fillna(data["Fare"].median())

# Pembagian data latih (70%) dan uji (30%)
train_df, test_df = train_test_split(
    data, test_size=0.30, random_state=42, stratify=data[target]
)
train_copy = train_df.copy()
test_copy = test_df.copy()

```

Tugas 4 > jawaban > 06_normalisasi_test_minmax.py > ...

```python

# Imputasi missing value Age menggunakan referensi nilai mean dari data latih
mean_age_train = train_copy.groupby(target)["Age"].mean()
overall_mean_train = train_copy["Age"].mean()

for cls in mean_age_train.index:
    mask_tr = (train_copy[target] == cls) & (train_copy["Age"].isna())
    train_copy.loc[mask_tr, "Age"] = mean_age_train[cls]
train_copy["Age"] = train_copy["Age"].fillna(overall_mean_train)

for cls in mean_age_train.index:
    mask_te = (test_copy[target] == cls) & (test_copy["Age"].isna())
    test_copy.loc[mask_te, "Age"] = mean_age_train[cls]
test_copy["Age"] = test_copy["Age"].fillna(overall_mean_train)

train_data = train_copy[features].astype(float)
test_data = test_copy[features].astype(float)

# Catat nilai minimum dan maksimum dari data latih (Langkah 5)
min_val = train_data.min()
max_val = train_data.max()

# 6. Normalisasi test_data menggunakan parameter min dan max dari Langkah 5
norm_test_data = (test_data - min_val) / (max_val - min_val)

print("=== 6. Normalisasi Min-Max 0-1 pada test_data (Parameter Train) ===")
print("Nilai Acuan Minimum (dari Data Latih):")
print(min_val)
print("\nNilai Acuan Maksimum (dari Data Latih):")
print(max_val)

sample_indices = test_data.sample(5, random_state=42).index

print("\n--- Sampel 5 Baris test_data Sebelum Normalisasi (Acak) ---")
print(test_data.loc[sample_indices])

print("\n--- Sampel 5 Baris test_data Setelah Normalisasi (Acak) ---")
print(norm_test_data.loc[sample_indices])
```

Melakukan normalisasi Min-Max pada ***test_data*** (30%) dengan menerapkan nilai minimum dan maksimum yang sebelumnya dicatat dari ***train_data*** (Langkah 5). Prosedur ini krusial untuk mencegah kebocoran data (***data leakage***) sehingga evaluasi model klasifikasi tetap objektif terhadap data baru.

HASIL LOAD DATASET

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> python 0
=== 6. Normalisasi Min-Max 0-1 pada test_data (Parameter Train) ===
Nilai Acuan Minimum (dari Data Latih):
Sex       0.00
Age       0.42
Pclass    1.00
Fare      0.00
dtype: float64

Nilai Acuan Maksimum (dari Data Latih):
Sex         1.0000
Age        80.0000
Pclass      3.0000
Fare      512.3292
dtype: float64

--- Sampel 5 Baris test_data Sebelum Normalisasi (Acak) ---
     Sex        Age  Pclass     Fare
511  1.0  31.355705     3.0   8.0500
329  0.0  16.000000     1.0  57.9792
216  0.0  27.000000     3.0   7.9250
322  0.0  30.000000     2.0  12.3500
830  0.0  15.000000     3.0  14.4542

--- Sampel 5 Baris test_data Setelah Normalisasi (Acak) ---
     Sex       Age  Pclass      Fare
511  1.0  0.388737     1.0  0.015713
329  0.0  0.195778     0.0  0.113168
216  0.0  0.334004     1.0  0.015469
322  0.0  0.371701     0.5  0.024106
830  0.0  0.183212     1.0  0.028213
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

7. KLASIFIKASI K-NN (K=3) & EVALUASI VALIDASI

Tugas 4 > jawaban > 07_knn_validation.py > ...

```python
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold, LeaveOneOut, train_test_split
from sklearn.neighbors import KNeighborsClassifier

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi fitur yang dibutuhkan serta target kelas
data = dataset[features + [target]].copy()
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})
data["Fare"] = data["Fare"].fillna(data["Fare"].median())
```

Tugas 4 > jawaban > 07_knn_validation.py > ...

```python

# Pipeline evaluasi mandiri per-split untuk mencegah kebocoran data (Data Leakage)
def evaluate_pipeline(df_train, df_test, k_neighbors=3):
    train_copy = df_train.copy()
    test_copy = df_test.copy()

    # Hitung rata-rata usia per kelas target hanya dari data latih
    mean_age_train = train_copy.groupby(target)["Age"].mean()
    overall_mean_train = train_copy["Age"].mean()

    # Terapkan imputasi pada data latih dan data uji dengan nilai mean data latih
    for cls in mean_age_train.index:
        mask_tr = (train_copy[target] == cls) & (train_copy["Age"].isna())
        train_copy.loc[mask_tr, "Age"] = mean_age_train[cls]
    train_copy["Age"] = train_copy["Age"].fillna(overall_mean_train)

    for cls in mean_age_train.index:
        mask_te = (test_copy[target] == cls) & (test_copy["Age"].isna())
        test_copy.loc[mask_te, "Age"] = mean_age_train[cls]
    test_copy["Age"] = test_copy["Age"].fillna(overall_mean_train)

    # Ekstraksi array fitur dan label target
    X_tr = train_copy[features].values.astype(float)
    y_tr = train_copy[target].values
    X_te = test_copy[features].values.astype(float)
    y_te = test_copy[target].values

    # Catat batas minimum dan maksimum dari data latih
    min_vals = np.min(X_tr, axis=0)
    max_vals = np.max(X_tr, axis=0)
    range_vals = np.where(max_vals - min_vals == 0, 1.0, max_vals - min_vals)

    # Normalisasi Min-Max [0, 1]
    X_tr_norm = (X_tr - min_vals) / range_vals
    X_te_norm = (X_te - min_vals) / range_vals

    # Latih model k-NN k=3 dan ukur tingkat kesalahan prediksi
    knn = KNeighborsClassifier(n_neighbors=k_neighbors)
    knn.fit(X_tr_norm, y_tr)
```

Membangun fungsi pipeline evaluasi klasifikasi menggunakan algoritma k-NN (k=3) yang mengintegrasikan pengisian missing value dan penskalaan Min-Max 0–1 berbasis parameter data latih di setiap split. Pipeline ini digunakan untuk memprediksi data uji pada seluruh metode validasi sekaligus menghitung nilai error ratio-nya.

7. KLASIFIKASI K-NN (K=3) & EVALUASI VALIDASI

Tugas 4 > jawaban > 07_knn_validation.py > ...

```python
def evaluate_pipeline(df_train, df_test, k_neighbors=3):

    y_pred = knn.predict(X_te_norm)
    error_ratio = np.mean(y_pred != y_te)

    return error_ratio, min_vals, max_vals


print("=" * 65)
print("=== 7. KLASIFIKASI k-NN (k=3) PADA SELURUH METODE VALIDASI ===")
print("=" * 65)

# --- 1. Hold-out Method (70% Train, 30% Test) ---
print("\n=== 1. HOLDOUT METHOD (70% - 30%) ===")
train_df, test_df = train_test_split(
    data, test_size=0.30, random_state=42, stratify=data[target]
)

err_holdout, min_holdout, max_holdout = evaluate_pipeline(train_df, test_df, k_neighbors=3)

print("Parameter Min Train (Sex, Age, Pclass, Fare):", np.round(min_holdout, 2))
print("Parameter Max Train (Sex, Age, Pclass, Fare):", np.round(max_holdout, 2))
print(f"Error Ratio : {err_holdout:.4f} ({err_holdout * 100:.2f}%)")
print(f"Akurasi     : {1 - err_holdout:.4f} ({(1 - err_holdout) * 100:.2f}%)")

# --- 2. K-Fold Cross Validation (k=10) ---
print("\n=== 2. K-FOLD CROSS VALIDATION (k=10) ===")
kf = KFold(n_splits=10, shuffle=True, random_state=42)
errors_kfold = []

for fold, (train_idx, test_idx) in enumerate(kf.split(data), 1):
    train_fold = data.iloc[train_idx]
    test_fold = data.iloc[test_idx]

    err, _, _ = evaluate_pipeline(train_fold, test_fold, k_neighbors=3)
    errors_kfold.append(err)
    print(f"  Fold {fold:02d} -> Error Ratio: {err:.4f} | Akurasi: {1 - err:.4f}")

mean_err_kfold = np.mean(errors_kfold)
print(f"\nRata-rata K-Fold Error Ratio : {mean_err_kfold:.4f} ({mean_err_kfold * 100:.2f}%)")
print(f"Rata-rata Akurasi            : {1 - mean_err_kfold:.4f} ({(1 - mean_err_kfold) * 100:.2f}%)")
```

Tugas 4 > jawaban > 07_knn_validation.py > ...

```python

# --- 3. Leave-One-Out (LOO) Cross Validation ---
print("\n=== 3. LEAVE-ONE-OUT (LOO) CROSS VALIDATION ===")
print("Memproses LOO (menjalankan 891 eksperimen)... Mohon tunggu...")
loo = LeaveOneOut()
errors_loo = []

for train_idx, test_idx in loo.split(data):
    train_loo = data.iloc[train_idx]
    test_loo = data.iloc[test_idx]

    err, _, _ = evaluate_pipeline(train_loo, test_loo, k_neighbors=3)
    errors_loo.append(err)

mean_err_loo = np.mean(errors_loo)
print(f"Total Eksperimen (N)          : {len(errors_loo)}")
print(f"Leave-One-Out Error Ratio     : {mean_err_loo:.4f} ({mean_err_loo * 100:.2f}%)")
print(f"Akurasi LOO                   : {1 - mean_err_loo:.4f} ({(1 - mean_err_loo) * 100:.2f}%)")

# --- Ringkasan Perbandingan Kinerja Seluruh Metode Validasi ---
print("\n" + "=" * 65)
print(f"{'Metode Validasi':<30} | {'Error Ratio':<15} | {'Akurasi':<12}")
print("-" * 65)
print(f"{'Hold-out Method (70% - 30%)':<30} | {err_holdout:<15.4f} | {1 - err_holdout:<12.4f}")
print(f"{'K-Fold Cross Validation (k=10)':<30} | {mean_err_kfold:<15.4f} | {1 - mean_err_kfold:<12.4f}")
print(f"{'Leave-One-Out (LOO)':<30} | {mean_err_loo:<15.4f} | {1 - mean_err_loo:<12.4f}")
print("=" * 65)
```

Mengeksekusi pengujian model klasifikasi k-NN (k=3) pada tiga skema validasi dari Hold-out Method (70%-30%), 10-Fold Cross Validation, dan Leave-One-Out (LOO). Setiap metode dihitung rasio kesalahannya (error ratio) serta tingkat akurasinya untuk membandingkan stabilitas performa model di berbagai skenario partisi dataset.

HASIL KLASIFIKASI K-NN (K=3) & EVALUASI VALIDASI

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> python 07_knn_val
============================================================
=== 7. KLASIFIKASI k-NN (k=3) PADA SELURUH METODE VALIDASI ===
============================================================

=== 1. HOLDOUT METHOD (70% - 30%) ===
Parameter Min Train (Sex, Age, Pclass, Fare): [0.   0.42 1.   0.  ]
Parameter Max Train (Sex, Age, Pclass, Fare): [  1.    80.     3.   512.33]
Error Ratio : 0.1343 (13.43%)
Akurasi     : 0.8657 (86.57%)

=== 2. K-FOLD CROSS VALIDATION (k=10) ===
  Fold 01 -> Error Ratio: 0.1444 | Akurasi: 0.8556
  Fold 02 -> Error Ratio: 0.1685 | Akurasi: 0.8315
  Fold 03 -> Error Ratio: 0.1236 | Akurasi: 0.8764
  Fold 04 -> Error Ratio: 0.1685 | Akurasi: 0.8315
  Fold 05 -> Error Ratio: 0.1124 | Akurasi: 0.8876
  Fold 06 -> Error Ratio: 0.1236 | Akurasi: 0.8764
  Fold 07 -> Error Ratio: 0.2022 | Akurasi: 0.7978
  Fold 08 -> Error Ratio: 0.1573 | Akurasi: 0.8427
  Fold 09 -> Error Ratio: 0.2247 | Akurasi: 0.7753
  Fold 10 -> Error Ratio: 0.1348 | Akurasi: 0.8652

Rata-rata K-Fold Error Ratio : 0.1560 (15.60%)
Rata-rata Akurasi            : 0.8440 (84.40%)

=== 3. LEAVE-ONE-OUT (LOO) CROSS VALIDATION ===
Memproses LOO (menjalankan 891 eksperimen)... Mohon tunggu...
Total Eksperimen (N)         : 891
Leave-One-Out Error Ratio    : 0.1582 (15.82%)
Akurasi LOO                  : 0.8418 (84.18%)

============================================================
Metode Validasi               | Error Ratio     | Akurasi
------------------------------------------------------------
Hold-out Method (70% - 30%)    | 0.1343          | 0.8657
K-Fold Cross Validation (k=10) | 0.1560          | 0.8440
Leave-One-Out (LOO)            | 0.1582          | 0.8418
============================================================
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

7A. EVALUASI MODEL - HOLDOUT METHOD

Tugas 4 > jawaban > 07a_knn_holdout.py > ...

```python
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi fitur yang dibutuhkan serta target kelas
data = dataset[features + [target]].copy()
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})
data["Fare"] = data["Fare"].fillna(data["Fare"].median())

# Pisahkan data latih (70%) dan data uji (30%) dengan stratified sampling agar proporsi kelas seimbang
train_df, test_df = train_test_split(
    data, test_size=0.30, random_state=42, stratify=data[target]
)

train_copy = train_df.copy()
test_copy = test_df.copy()

# Imputasi missing value Age menggunakan nilai mean per kelas dari data latih (mencegah data leakage)
mean_age_train = train_copy.groupby(target)["Age"].mean()
overall_mean_train = train_copy["Age"].mean()

for cls in mean_age_train.index:
    mask_train = (train_copy[target] == cls) & (train_copy["Age"].isna())
    train_copy.loc[mask_train, "Age"] = mean_age_train[cls]
train_copy["Age"] = train_copy["Age"].fillna(overall_mean_train)

for cls in mean_age_train.index:
    mask_test = (test_copy[target] == cls) & (test_copy["Age"].isna())
    test_copy.loc[mask_test, "Age"] = mean_age_train[cls]
test_copy["Age"] = test_copy["Age"].fillna(overall_mean_train)

# Ekstraksi matriks fitur dan vektor target
X_tr = train_copy[features].values.astype(float)
y_tr = train_copy[target].values
X_te = test_copy[features].values.astype(float)
y_te = test_copy[target].values
```

Tugas 4 > jawaban > 07a_knn_holdout.py > ...

```python

# Catat nilai min dan max dari data latih untuk normalisasi Min-Max
min_vals = np.min(X_tr, axis=0)
max_vals = np.max(X_tr, axis=0)
range_vals = np.where(max_vals - min_vals == 0, 1.0, max_vals - min_vals)

# Terapkan penskalaan rentang 0-1 pada data latih dan data uji dengan parameter acuan data latih
X_tr_norm = (X_tr - min_vals) / range_vals
X_te_norm = (X_te - min_vals) / range_vals

# Latih model klasifikasi k-NN dengan jumlah tetangga terdekat k=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_tr_norm, y_tr)

# Prediksi label kelas pada data uji dan hitung tingkat kesalahan klasifikasi (Error Ratio)
y_pred = knn.predict(X_te_norm)
error_ratio = np.mean(y_pred != y_te)
accuracy = 1.0 - error_ratio

print("=== 7.a Klasifikasi k-NN (k=3) dengan Hold-out Method (70% - 30%) ===")
print(f"Jumlah Data Latih : {len(train_df)} baris")
print(f"Jumlah Data Uji   : {len(test_df)} baris\n")

print("--- Parameter Min & Max Acuan Data Latih ---")
for feat, mi, ma in zip(features, min_vals, max_vals):
    print(f"  {feat:<10}: Min = {mi:.2f}, Max = {ma:.2f}")

print("\n--- Hasil Evaluasi Kinerja Model ---")
print(f"Error Ratio : {error_ratio:.4f} ({error_ratio * 100:.2f}%)")
print(f"Akurasi     : {accuracy:.4f} ({accuracy * 100:.2f}%)")

# Ambil sampel acak menggunakan index yang sama untuk membandingkan prediksi vs label aktual
rng = np.random.RandomState(42)
sample_eval_idx = rng.choice(len(y_te), size=10, replace=False)

eval_sample_df = pd.DataFrame({
    "Label Aktual": y_te[sample_eval_idx],
    "Hasil Prediksi": y_pred[sample_eval_idx],
    "Status": np.where(y_te[sample_eval_idx] == y_pred[sample_eval_idx], "Tepat", "Salah")
})

print("\n--- Sampel 10 Hasil Prediksi Data Uji (Acak) ---")
print(eval_sample_df.to_string(index=False))

```

Mengimplementasikan evaluasi model k-NN (k=3) menggunakan metode validasi Hold-out (70% data latih, 30% data uji) secara stratified. Proses imputasi missing value dan normalisasi Min-Max 0–1 pada data uji sepenuhnya merujuk pada statistik data latih guna mencegah kebocoran data (data leakage), diakhiri dengan penghitungan metrik error ratio dan akurasi.

HASIL
EVALUASI MODEL - HOLDOUT METHOD

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> python 07a
=== 7.a Klasifikasi k-NN (k=3) dengan Hold-out Method (70% - 30%) ===
Jumlah Data Latih : 623 baris
Jumlah Data Uji   : 268 baris

--- Parameter Min & Max Acuan Data Latih ---
  Sex       : Min = 0.00, Max = 1.00
  Age       : Min = 0.42, Max = 80.00
  Pclass    : Min = 1.00, Max = 3.00
  Fare      : Min = 0.00, Max = 512.33

--- Hasil Evaluasi Kinerja Model ---
Error Ratio : 0.1343 (13.43%)
Akurasi     : 0.8657 (86.57%)

--- Sampel 10 Hasil Prediksi Data Uji (Acak) ---
 Label Aktual  Hasil Prediksi Status
            0               0  Tepat
            1               1  Tepat
            1               1  Tepat
            1               1  Tepat
            1               1  Tepat
            0               0  Tepat
            1               1  Tepat
            0               0  Tepat
            1               1  Tepat
            1               1  Tepat
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

8. KLASIFIKASI K-NN: K-FOLD CROSS VALIDATION (K=10)

Tugas 4 > jawaban > 07b_knn_kfold.py > ...

```python
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
from sklearn.neighbors import KNeighborsClassifier

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi fitur dan target
data = dataset[features + [target]].copy()
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})
data["Fare"] = data["Fare"].fillna(data["Fare"].median())

def evaluate_fold(df_train, df_test, k_neighbors=3):
    train_copy = df_train.copy()
    test_copy = df_test.copy()

    # Hitung rata-rata usia per kelas pada data latih lipatan ini
    mean_age_train = train_copy.groupby(target)["Age"].mean()
    overall_mean_train = train_copy["Age"].mean()

    # Imputasi missing value Age dengan parameter dari lipatan train
    for cls in mean_age_train.index:
        mask_tr = (train_copy[target] == cls) & (train_copy["Age"].isna())
        train_copy.loc[mask_tr, "Age"] = mean_age_train[cls]
    train_copy["Age"] = train_copy["Age"].fillna(overall_mean_train)

    for cls in mean_age_train.index:
        mask_te = (test_copy[target] == cls) & (test_copy["Age"].isna())
        test_copy.loc[mask_te, "Age"] = mean_age_train[cls]
    test_copy["Age"] = test_copy["Age"].fillna(overall_mean_train)

    X_tr = train_copy[features].values.astype(float)
    y_tr = train_copy[target].values
    X_te = test_copy[features].values.astype(float)
    y_te = test_copy[target].values
```

Tugas 4 > jawaban > 07b_knn_kfold.py > ...

```python
def evaluate_fold(df_train, df_test, k_neighbors=3):
    # Penskalaan fitur Min-Max dengan parameter acuan data latih lipatan ini
    min_vals = np.min(X_tr, axis=0)
    max_vals = np.max(X_tr, axis=0)
    range_vals = np.where(max_vals - min_vals == 0, 1.0, max_vals - min_vals)

    X_tr_norm = (X_tr - min_vals) / range_vals
    X_te_norm = (X_te - min_vals) / range_vals

    # Latih model k-NN k=3 dan ukur tingkat kesalahan klasifikasi
    knn = KNeighborsClassifier(n_neighbors=k_neighbors)
    knn.fit(X_tr_norm, y_tr)

    y_pred = knn.predict(X_te_norm)
    return np.mean(y_pred != y_te)

# Inisialisasi skema 10-Fold Cross Validation dengan pengacakan terkontrol
kf = KFold(n_splits=10, shuffle=True, random_state=42)
errors_kfold = []

print("=== 7.b Klasifikasi k-NN (k=3) dengan K-Fold Cross Validation (k=10) ===")
print(f"{'Fold':<8} | {'Jumlah Train':<14} | {'Jumlah Test':<13} | {'Error Ratio':<13} | {'Akurasi':<10}")
print("-" * 65)

for fold, (train_idx, test_idx) in enumerate(kf.split(data), 1):
    train_fold = data.iloc[train_idx]
    test_fold = data.iloc[test_idx]

    err = evaluate_fold(train_fold, test_fold, k_neighbors=3)
    errors_kfold.append(err)
    print(f"Fold {fold:02d}   | {len(train_fold):<14} | {len(test_fold):<13} | {err:<13.4f} | {1 - err:<10.4f}")

mean_err = np.mean(errors_kfold)
mean_acc = 1.0 - mean_err

print("-" * 65)
print(f"Rata-rata K-Fold Error Ratio : {mean_err:.4f} ({mean_err * 100:.2f}%)")
print(f"Rata-rata K-Fold Akurasi     : {mean_acc:.4f} ({mean_acc * 100:.2f}%)")
```

Mengimplementasikan evaluasi klasifikasi k-NN (k=3) menggunakan skema 10-Fold Cross Validation. Di setiap iterasi lipatan, imputasi nilai hilang dan penskalaan Min-Max diisolasi hanya pada data latih fold tersebut. Metode ini memastikan seluruh data sampel digunakan secara bergantian sebagai data latih dan data uji, menghasilkan estimasi error ratio rata-rata yang lebih stabil dan objektif.

HASIL

KLASIFIKASI K-NN: K-FOLD CROSS VALIDATION (K=10)

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> python 07b_knn_
=== 7.b Klasifikasi k-NN (k=3) dengan K-Fold Cross Validation (k=10) ===
Fold     | Jumlah Train   | Jumlah Test   | Error Ratio   | Akurasi
-----------------------------------------------------------------
Fold 01  | 801            | 90            | 0.1444        | 0.8556
Fold 02  | 802            | 89            | 0.1685        | 0.8315
Fold 03  | 802            | 89            | 0.1236        | 0.8764
Fold 04  | 802            | 89            | 0.1685        | 0.8315
Fold 05  | 802            | 89            | 0.1124        | 0.8876
Fold 06  | 802            | 89            | 0.1236        | 0.8764
Fold 07  | 802            | 89            | 0.2022        | 0.7978
Fold 08  | 802            | 89            | 0.1573        | 0.8427
Fold 09  | 802            | 89            | 0.2247        | 0.7753
Fold 10  | 802            | 89            | 0.1348        | 0.8652
-----------------------------------------------------------------
Rata-rata K-Fold Error Ratio : 0.1560 (15.60%)
Rata-rata K-Fold Akurasi     : 0.8440 (84.40%)
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

7C. KLASIFIKASI K-NN: LEAVE-ONE-OUT (LOO)

Tugas 4 > jawaban > 07c_knn_loo.py > ...

```python
import os
import time
import numpy as np
import pandas as pd
from sklearn.model_selection import LeaveOneOut
from sklearn.neighbors import KNeighborsClassifier

base_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
if not os.path.exists(csv_path):
    csv_path = 'data/titanic.csv'
if not os.path.exists(csv_path):
    csv_path = 'titanic.csv'

dataset = pd.read_csv(csv_path)

features = ["Sex", "Age", "Pclass", "Fare"]
target = "Survived"

# Ekstraksi fitur dan target
data = dataset[features + [target]].copy()
data["Sex"] = data["Sex"].map({"female": 0, "male": 1})
data["Fare"] = data["Fare"].fillna(data["Fare"].median())

# Inisialisasi Leave-One-Out di mana setiap iterasi menguji tepat 1 sampel
loo = LeaveOneOut()
total_samples = len(data)
errors_loo = []

print("=== 7.c Klasifikasi k-NN (k=3) dengan Leave-One-Out (LOO) ===")
print(f"Total Sampel Data (N) : {total_samples}")
print("Memproses LOO (menjalankan 891 pengujian satu per satu)...")

start_time = time.time()

for idx, (train_idx, test_idx) in enumerate(loo.split(data), 1):
    train_copy = data.iloc[train_idx].copy()
    test_copy = data.iloc[test_idx].copy()

    # Hitung rata-rata usia per kelas pada data latih (N - 1 sampel)
    mean_age_train = train_copy.groupby(target)["Age"].mean()
    overall_mean_train = train_copy["Age"].mean()

    # Imputasi missing value Age dengan parameter dari data latih
    for cls in mean_age_train.index:
        mask_tr = (train_copy[target] == cls) & (train_copy["Age"].isna())
        train_copy.loc[mask_tr, "Age"] = mean_age_train[cls]
    train_copy["Age"] = train_copy["Age"].fillna(overall_mean_train)
```

Tugas 4 > jawaban > 07c_knn_loo.py > ...

```python
    for cls in mean_age_train.index:
        mask_te = (test_copy[target] == cls) & (test_copy["Age"].isna())
        test_copy.loc[mask_te, "Age"] = mean_age_train[cls]
    test_copy["Age"] = test_copy["Age"].fillna(overall_mean_train)

    X_tr = train_copy[features].values.astype(float)
    y_tr = train_copy[target].values
    X_te = test_copy[features].values.astype(float)
    y_te = test_copy[target].values

    # Penskalaan fitur Min-Max dengan acuan data latih
    min_vals = np.min(X_tr, axis=0)
    max_vals = np.max(X_tr, axis=0)
    range_vals = np.where(max_vals - min_vals == 0, 1.0, max_vals - min_vals)

    X_tr_norm = (X_tr - min_vals) / range_vals
    X_te_norm = (X_te - min_vals) / range_vals

    # Latih model k-NN k=3 dan catat ketepatan prediksi pada 1 sampel uji
    knn = KNeighborsClassifier(n_neighbors=3)
    knn.fit(X_tr_norm, y_tr)

    y_pred = knn.predict(X_te_norm)
    errors_loo.append(int(y_pred[0] != y_te[0]))

    # Tampilkan pembaruan progres berkala ke terminal
    if idx % 150 == 0 or idx == total_samples:
        print(f"  Progres: {idx}/{total_samples} sampel selesai dievaluasi...")

duration = time.time() - start_time
mean_err_loo = np.mean(errors_loo)
accuracy_loo = 1.0 - mean_err_loo

print("\n--- Hasil Evaluasi Leave-One-Out (LOO) ---")
print(f"Waktu Eksekusi            : {duration:.2f} detik")
print(f"Total Prediksi Salah      : {np.sum(errors_loo)} dari {total_samples}")
print(f"Leave-One-Out Error Ratio : {mean_err_loo:.4f} ({mean_err_loo * 100:.2f}%)")
print(f"Leave-One-Out Akurasi     : {accuracy_loo:.4f} ({accuracy_loo * 100:.2f}%)")
```

Mengimplementasikan evaluasi klasifikasi k-NN (k=3) dengan metode Leave-One-Out (LOO) Cross Validation melalui 891 eksperimen independen. Pada setiap iterasi, satu sampel diuji menggunakan model yang dilatih dan dinormalisasi murni dari N-1 data latih yang tersisa, menghasilkan estimasi performa (error ratio) dengan tingkat bias yang sangat rendah terhadap data baru.

HASIL
KLASIFIKASI K-NN: LEAVE-ONE-OUT (LOO)

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban> py
=== 7.c Klasifikasi k-NN (k=3) dengan Leave-One-Out (LOO) ===
Total Sampel Data (N) : 891
Memproses LOO (menjalankan 891 pengujian satu per satu)...
  Progres: 150/891 sampel selesai dievaluasi...
  Progres: 300/891 sampel selesai dievaluasi...
  Progres: 450/891 sampel selesai dievaluasi...
  Progres: 600/891 sampel selesai dievaluasi...
  Progres: 750/891 sampel selesai dievaluasi...
  Progres: 891/891 sampel selesai dievaluasi...

--- Hasil Evaluasi Leave-One-Out (LOO) ---
Waktu Eksekusi            : 8.43 detik
Total Prediksi Salah      : 141 dari 891
Leave-One-Out Error Ratio : 0.1582 (15.82%)
Leave-One-Out Akurasi     : 0.8418 (84.18%)
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 4\jawaban>
```

KESIMPULAN

Eksplorasi terhadap model klasifikasi k-NN (k=3) pada dataset Titanic menunjukkan bahwa hasil evaluasi sangat dipengaruhi oleh strategi partisi data dan isolasi tahapan preprocessing. Penanganan missing value usia berbasis kelas target serta normalisasi Min-Max yang dihitung murni dari data latih terbukti efektif menjaga integritas data uji dari kebocoran informasi (data leakage).

- Dampak Isolasi Preprocessing terhadap Validitas Model: Melakukan imputasi dan penskalaan fitur secara terpisah di setiap iterasi split memastikan model dievaluasi secara realistis dan objektif terhadap data yang benar-benar belum pernah dilihat sebelumnya.
- Trade-off Kinerja Ketiga Metode Validasi: Metode Hold-out (70:30) unggul dalam kecepatan eksekusi namun berisiko bias terhadap pemisahan acak, K-Fold (k=10) terbukti paling ideal dan stabil dalam menyeimbangkan bias serta variansi, sementara Leave-One-Out (LOO) memberikan estimasi galat paling presisi dengan konsekuensi beban komputasi N iterasi yang paling berat.

TUGAS M5 - DATA MINING

TERIMA KASIH

You can access my GitHub repository for the source code :

https://github.com/GusthiPangestu1906/WDM.git